

AIR SENTINEL

Code Learning Handbook

Understand the AirSentinel code well enough to explain and extend it

Purpose

This handbook is a beginner's map from Python files to the working product. It focuses on how the real repository works, not on memorising every line.

What is inside

- A 2–4 hour preparation route
- What every folder does
- How data becomes a forecast and a case
- Security, Google adapters and testing
- Practical questions judges may ask

Prepared from the verified local AirSentinel repository. It clearly separates what already works from work that needs your team's accounts, approvals or field validation.

1. Learn the system in the right order

Time	Study	Result
0–30 minutes	README.md, HANDOFF.md and the public dashboard	You can state the goal, non-claims and user roles.
30–75 minutes	Data scripts and processed CSV outputs	You know where observations come from and why provenance matters.
75–120 minutes	Features, model training and forecast output	You can explain lags, chronological tests, MAE/RMSE and uncertainty.
120–165 minutes	Anomaly/case pipeline and authority app	You can explain human review and audit safety.
165–240 minutes	Cloud adapters, deployment guide and tests	You can distinguish local working code from account-bound cloud activation.

Best learning method

Run a small command, open the file that produced the output, and answer three questions: what is the input, what transformation happens, and what output is saved? This is better than trying to read the entire codebase from top to bottom.

2. The project map

Path	Plain-English purpose
app.py	Public aggregate dashboard. It shows reviewed, non-sensitive information.
citizen_app.py	Consent-based citizen text/photo/voice reporting interface.
authority_app.py	Reviewer dashboard. Local prototype actions are audited; production mode is view-only.

Path	Plain-English purpose
src/api/	FastAPI backend, request schemas and access checks for production-like protected actions.
src/data/	Downloads, normalises, validates and reports station/pollutant data.
src/features/	Builds time-series features such as lag values and weather variables.
src/models/	Trains, compares, predicts and documents conventional locality models.
src/scoring/	Turns an unexpected forecast mismatch into a review case—not a source claim.
src/authority/ and src/services/	Case queue, audit record, citizen report and guarded Gemini helpers.
src/cloud/	Feature-flagged adapters for Firebase, BigQuery, Gemini, Maps, Earth Engine and language services.
deploy/	Cloud Run/GCP bootstrap and safe packaging scripts.
tests/	Automated checks proving safety, source labels, access and data gates.

3. Python ideas you are using

Idea	Where it appears	Why it matters
Modules and imports	Every src/ folder	Lets the app reuse one reliable function instead of duplicating logic.
Pathlib Path	Data and model scripts	Builds safe Windows/Linux file paths such as data/processed/output.csv.
Functions	fetch_*, build_*, train_*	A named, testable unit: input → work → output.
Pandas DataFrame	Data/feature/model scripts	A spreadsheet-like table that can filter, join, group and save CSV.
Requests	Data connectors	Calls an API and receives JSON, with timeout/error handling.
Environment variables	.env / cloud adapters	Keep secrets outside source code; feature flags make optional services safe.
Pydantic schemas	src/api/schemas.py	Check that API requests have the expected fields and types.
Tests	tests/	Verify assumptions repeatedly instead of trusting a manual demo.

4. How data becomes an AirSentinel output

The core pipeline

- 1. Discover** — Find nearby stations for cities/locality pilot anchors and the six target pollutants.
- 2. Fetch** — Download latest/historical observations and weather with pagination, retries and provenance.
- 3. Validate** — Reject malformed records, standardise units/time, deduplicate and quantify coverage gaps.

- 4. Build features** — Make lag/rolling/time/weather features from prior observations only.
- 5. Evaluate** — Train a conventional model on earlier time; test it on later time to avoid leakage.
- 6. Predict** — Create 1h, 3h and 4h historical prototype forecast candidates with empirical intervals.
- 7. Review** — If actual data later disagrees strongly, create a cause-unverified case for a person to review.

```
.\.venv\Scripts\python.exe src\run_locality_model_and_review_refresh.py
```

This orchestration command runs the locality preparation/model/review sequence. It creates processed files. It does not turn a historical result into a live official alert.

5. Data scripts: learn input → output

File	Input	Output / learning point
fetch_multiscale_pollutant_history.py	OpenAQ fallback or configured source; area/pollutant/date arguments	Historical multi-pollutant observations with source labels.
fetch_cpcb_realtime_snapshot.py	Authorised CPCB/data.gov.in key	Current official snapshot when access exists; otherwise safely disabled.
validate_observation_contract.py	Raw observation table	Checks canonical fields, pollutants, timestamps and provenance.
report_multiscale_coverage.py	Processed history	Rows, span, gaps, stations and completeness by area/pollutant.
build_locality_evidence_profile.py	Coverage and station data	Explains whether a locality has enough evidence for modelling/review.

Why six pollutants?

PM2.5 is the main early forecast target in this prototype, while PM10, NO₂, O₃, SO₂ and CO provide

broader air-quality context. Do not claim that all six are independently forecast or that a single number is an official AQI.

6. Feature engineering: the line of thinking

A model cannot predict the next hour from future information. Feature engineering creates columns that were available at the prediction time.

Feature	Example	Reason
Lag	pm25_lag_1h = PM2.5 measured one hour earlier	Air quality has time continuity.
Rolling mean	mean of previous 3 or 6 hours	Reduces the effect of one noisy observation.
Time features	hour and day_of_week	Captures patterns like rush-hour or weekly changes.
Weather	temperature, humidity, wind, rainfall, pressure	Weather can influence pollutant dispersion and persistence.
Target	pm25_next_1h / next_3h / next_4h	The future value used only for supervised training/evaluation.

Read `src/features/build_locality_multi_horizon_features.py`. The most important safety rule is that all lag/rolling features look backwards in time. The file excludes rows with missing requirements rather than silently inventing values.

7. ML in simple language

Term	Meaning in AirSentinel
Persistence baseline	A simple comparison: predict the next value will be the current value. Any conventional model must beat it to be useful.

Term	Meaning in AirSentinel
Conventional model	A non-deep-learning baseline model using tabular lag/weather features. The project deliberately does not promote deep or federated learning.
Chronological split	Earlier data is training; later data is testing. This imitates real forecasting and prevents future leakage.
MAE	Average absolute forecast error. Lower is better and easy to explain in $\mu\text{g}/\text{m}^3$.
RMSE	Error metric that penalises larger mistakes more strongly. Lower is better.
Empirical interval	A historically derived uncertainty range around the prediction. It is not a guarantee.
Promotion gate	A quality and comparison check. A model does not become operational only because it produces a number.

```
.\.venv\Scripts\python.exe src\models\train_locality_multi_horizon.py
```

```
.\.venv\Scripts\python.exe src\models\predict_locality_multi_horizon.py
```

What to say to judges

‘We use a conservative conventional time-series baseline with chronological hold-out evaluation. We show uncertainty and keep new localities gated until they have enough aligned data. We do not claim deep learning or federated learning.’

8. Sudden spike: why the model does not ‘fail silently’

Suppose a model expects PM2.5 around 28 but a later verified value is 38. The difference is called a residual. The anomaly pipeline measures whether a residual is unusually large compared with historical residuals. If it is, it creates a review case.

The code does	The code deliberately does not do
Keeps a residual and robust residual score.	Claim that stubble burning, an industry or a person caused the spike.
Requests corroboration: fresh stations, weather and moderated community context.	Automatically issue enforcement, public health or source-attribution instructions.
Adds the case to a queue with an owner/status/action fields.	Treat a photo, purifier or citizen report as a calibrated outdoor monitor.

```
.\.venv\Scripts\python.exe src\scoring\build_locality_anomaly_cases.py
```

```
.\.venv\Scripts\python.exe src\authority\build_authority_case_queue.py
```

9. Dashboards, API and safety boundary

Surface	Who uses it	What it is allowed to do
app.py	Public/judges	Show aggregate reviewed information and explanation of uncertainty.
citizen_app.py	A consented public user	Create coarse evidence for pending moderation; no claim of measurement truth.
authority_app.py	Authorised reviewer	Review queue and audited action workflow; production is deliberately view-only.

Surface	Who uses it	What it is allowed to do
src/api/main.py	Protected service-to-service/client calls	Validate identity/schema, make controlled durable actions, audit them.

FastAPI is the backend framework. Streamlit is the dashboard framework. A browser talks to Streamlit for screens; protected services talk to FastAPI for controlled actions. API schemas prevent malformed payloads; security checks refuse an unauthorised role.

10. Google integrations: understand the adapter pattern

The src/cloud and src/services folders use guarded adapters. That means a feature is disabled unless the required environment configuration exists. This avoids a dashboard pretending a provider call succeeded when the project/key has not been configured.

Adapter	Real job	Safe rule
Gemini explainer/photo	Summarise bounded evidence or describe consented photo context.	No AQI, cause, enforcement, health or liability decision.
Firebase store	Hold pending reviewed citizen evidence.	Verified identity, deny-by-default rules, retention and moderation.
BigQuery loader	Store approved analytical/audit tables.	Least-privilege roles; no raw personal reports.
Maps / Earth Engine	Show locality/satellite context.	Satellite column/context layer is not surface PM2.5 proof.
Language services	Translate or transcribe consented reports.	Keep consent, language QA and human review.
Vertex readiness	Possible managed hosting path after evaluation.	Not a claim that a superior deep model is deployed.

11. How to test before saying it works

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

```
.\.venv\Scripts\python.exe -m compileall -q src app.py authority_app.py citizen_app.py
```

The tests cover API access, audit safety, cloud integration safety, source/promotion gates and long-history ingestion. A test passing is evidence that a defined expected behaviour remains intact; it is not proof that a real government deployment is approved.

Common issue	Meaning	First response
'Path not found'	Command Prompt is not inside the project folder.	Run cd C:\Users\Rupesh\AirSentinel\AirSentinel first.
'not recognised'	A Python script was typed without python.exe or from the wrong folder.	Use .\.venv\Scripts\python.exe followed by the script path.
Dashboard port busy	Another Streamlit process owns 8501/8502/8503.	Stop it with Ctrl+C or use a different --server.port.
Disabled Google adapter	No approved project/key/feature flag was supplied.	This is safe. Configure through Secret Manager/GCP, not by hard-coding a key.

12. Ten questions you should be ready to answer

What problem is solved? A city-to-locality evidence, forecast and review gap—not an attempt to replace the regulator.

Why not city average only? A city average can hide a local industrial edge, village boundary or corridor event.

Why 1h/3h/4h? They give short operational lead time while keeping the research scope conservative.

Why not use deep learning? Data quality, locality coverage and explainability are more important than a complex model claim. The team does not promote unvalidated deep learning.

How handle a surprise spike? Compare later observation to prediction, create a residual case, request corroboration and require human review.

Can you identify the source? No. Evidence can support triage but source attribution requires official investigation.

What does Gemini do? Guarded language/photo context assistance only; it cannot issue AQI, cause or enforcement decisions.

What is your data fallback? OpenAQ development fallback, visibly labelled; official CPCB/data.gov.in path is preferred when authorised.

How protect users? Coarse consented reports, verified identity in production, deny-by-default rules, moderation, retention and audit logs.

What must happen before pilot use? Official/fresh data, station/locality validation, identity and persistence tests, human workflow approval and cloud governance.

13. Your practical study checklist

1. Run the public dashboard and explain every section in your own words.
2. Open one CSV in data/processed and trace the producing script.
3. Read src/features/build_locality_multi_horizon_features.py and identify two lags and one weather feature.
4. Read src/models/train_locality_multi_horizon.py and find the chronological train/test split and metrics.
5. Read src/scoring/build_locality_anomaly_cases.py and explain why cause_status stays 'unverified'.
6. Read src/api/security.py and explain why protected endpoints need a role.
7. Run tests and say what passing tests mean—and what they do not prove.